Instituto Federal de Brasília - IFB
Campus: Brasília
Curso: Tecnologia em Sistemas para Internet
Componente: Práticas de Desenvolvimento de Software - 2026/2
Docente: Prof. Dr. Dauster Souza Pereira

Demonstração GitHub Actions - Atividade Prática sobre TDD

Discentes:
Lucas Nunes Gomes - 251016600011
Wilson de Brito Ferreira - 251016600007

Relatório da Atividade

Exercício 1: Calculadora de Desconto

1. **Qual foi o primeiro teste criado?**

   O primeiro teste criado foi a função `test_calcular_desconto_10()`, responsável por validar o desconto de 10% sobre o valor de 100, esperando como resultado o valor 90.

2. **Por que ele falhou?**

   O teste falhou por asserção (*AssertionError*), pois na fase RED a função `calcular_desconto(valor, percentual)` no módulo `calculadora.py` apenas definia um esqueleto de método com retorno `None`, resultando na falha `assert None == 90` mostrada pelo *Pytest*.

3. **Como você fez o teste passar?**

   A fórmula matemática mínima necessária foi implmentada em `calculadora.py`: o cálculo do valor final descontado através

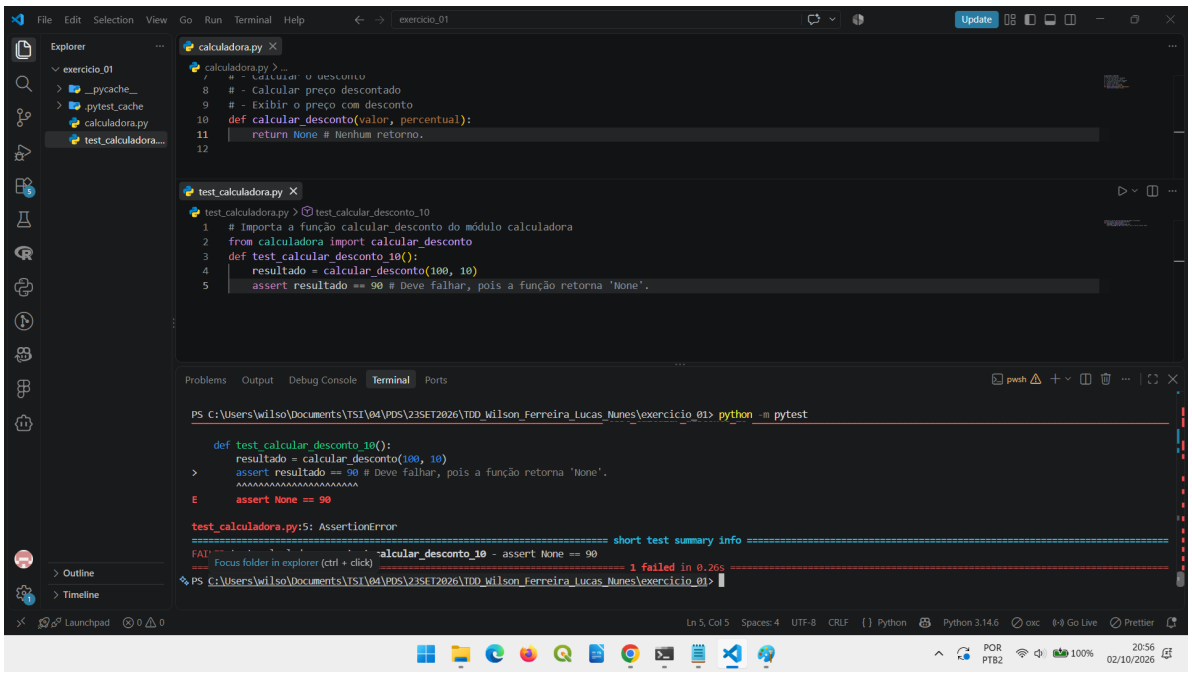


Figura 1: Evidência da fase RED: falha de asserção no Pytest.

da expressão `valor * (1 - percentual / 100)`, satisfazendo a asserção e fazendo a execução verde no *Pytest* (fase GREEN).

4. **O que foi alterado durante a refatoração?**

   Na fase REFACTOR, o conjunto de testes foi expandido para cobrir todos os cenários obrigatórios (20%, 0% e 100%) da questão e os casos do desafio adicional (preço negativo e desconto superior a 100%). Em `calculadora.py`, foram adicionadas condições de guarda com lançamento de exceções (`raise ValueError`), validadas via `pytest.raises(ValueError)`, confirmando os 6 testes passando normalmente.

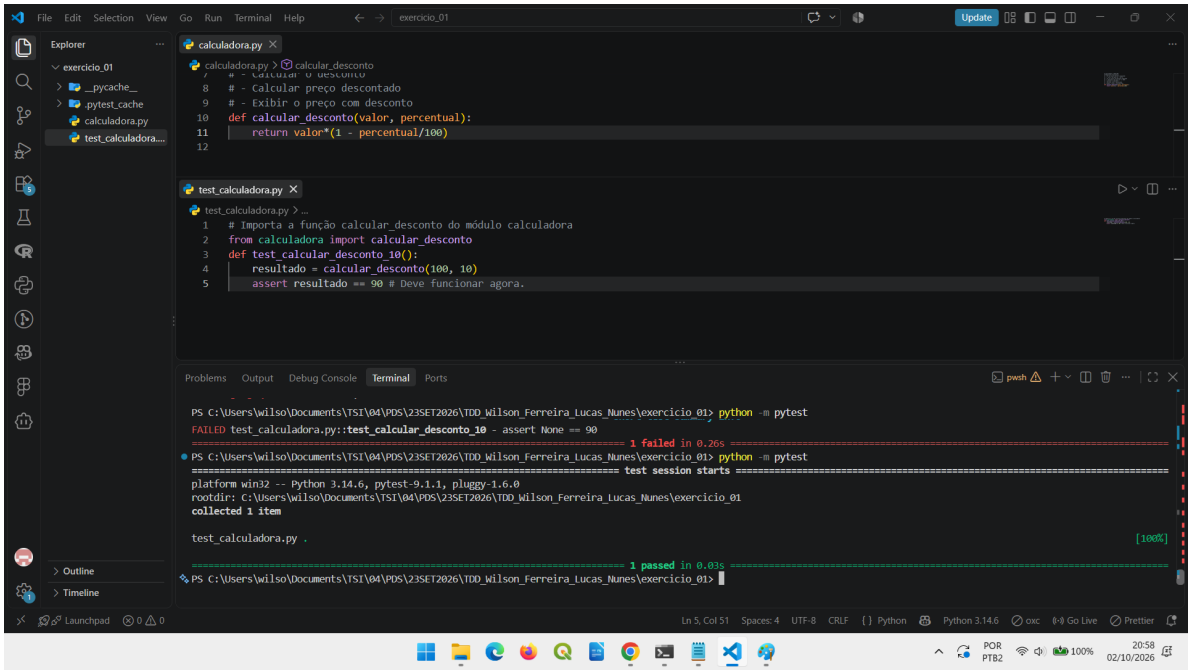


Figura 2: Evidência da fase GREEN: teste obrigatório aprovado.

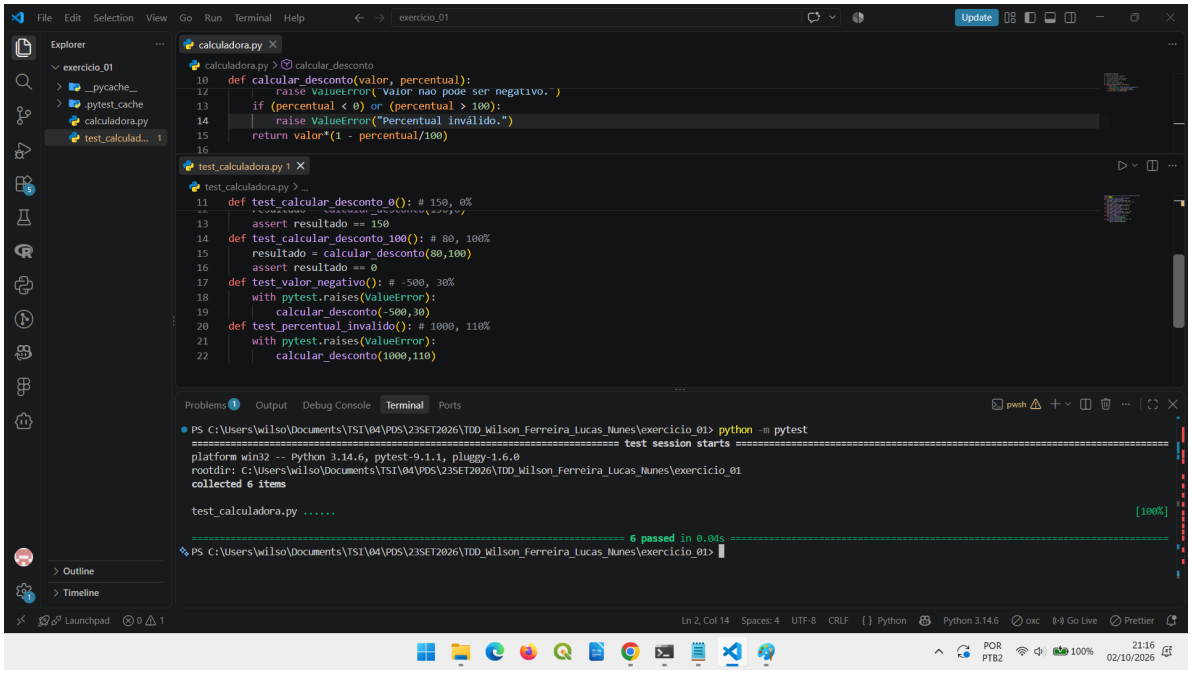


Figura 3: Evidência da fase REFACTOR: 6 testes aprovados.

Exercício 2: Sistema de Classificação de Notas

1. **Quais testes foram criados?**

   Foram concebidos e executados 7 testes unitários automatizados com recurso ao *Pytest* no módulo `test_notas.py`, cobrindo todos os cenários obrigatórios e os casos de exceção:

   - `test_nota_quatro_reprovado()`: valida se a nota 4 retorna a classificação "Reprovado".
   - `test_nota_cinco_recuperacao()`: valida o limite inferior da faixa de recuperação com a nota 5.
   - `test_nota_seis_nove_recuperacao()`: valida o limite superior da faixa de recuperação através da nota de fronteira 6,9.
   - `test_nota_sete_nove_aprovado()`: valida o limiar de aprovação através da nota 7.
   - `test_nota_dez_aprovado()`: valida a pontuação máxima da escala com a nota 10.
   - `test_nota_negativa_invalida()`: valida o lançamento de exceção para valores estritamente negativos (ex.: -15).
   - `test_nota_maior_que_dez_invalida()`: valida o lançamento de exceção para valores que excedem a escala (ex.: 15).

2. **Qual caso de teste apresentou maior dificuldade?**

   A maior exigência técnica residiu na validação dos casos de fronteira (*boundary value analysis*), com particular destaque para o limite superior da faixa de recuperação na nota 6,9 em transição direta para a nota 7 (aprovação). Foi necessário garantir que a lógica condicional em `notas.py` operasse com operadores relacionais adequados (`nota < 7`), evitando desvios com números decimais. Paralelamente, a estruturação dos testes para verificar o disparo e a captura correta de exceções demandou o uso rigoroso do gestor de contexto `pytest.raises`.

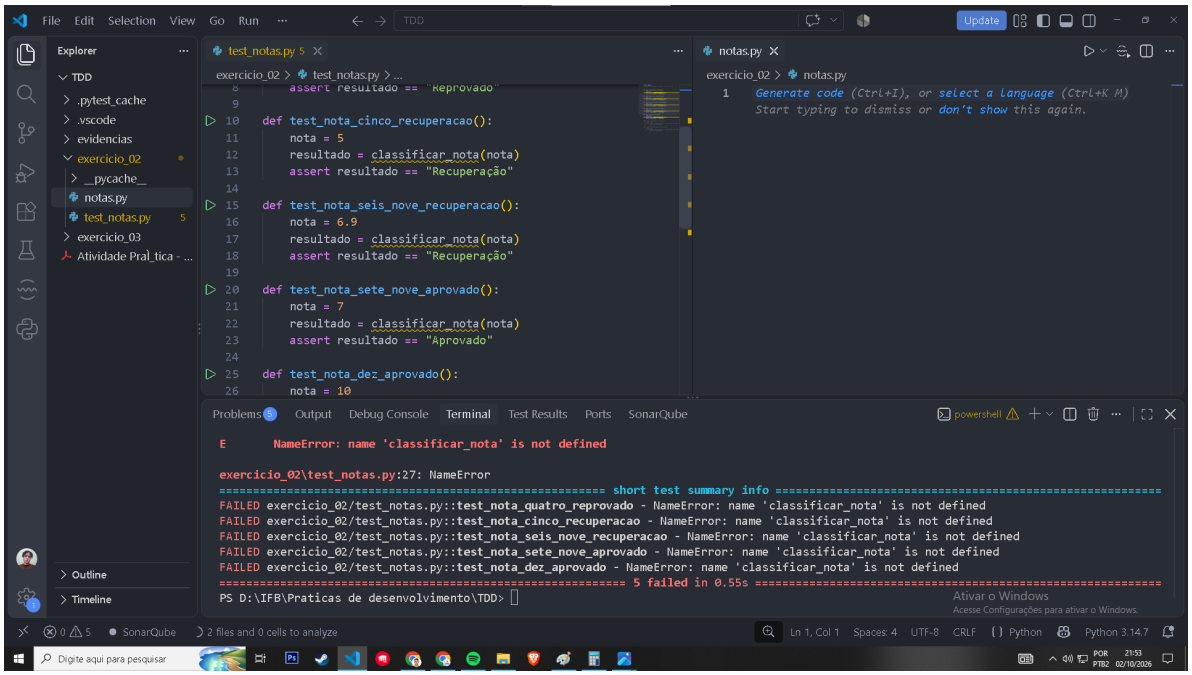


Figura 4: Evidência da fase RED: falha inicial no Pytest (*NameError*) antes da definição da função no módulo.

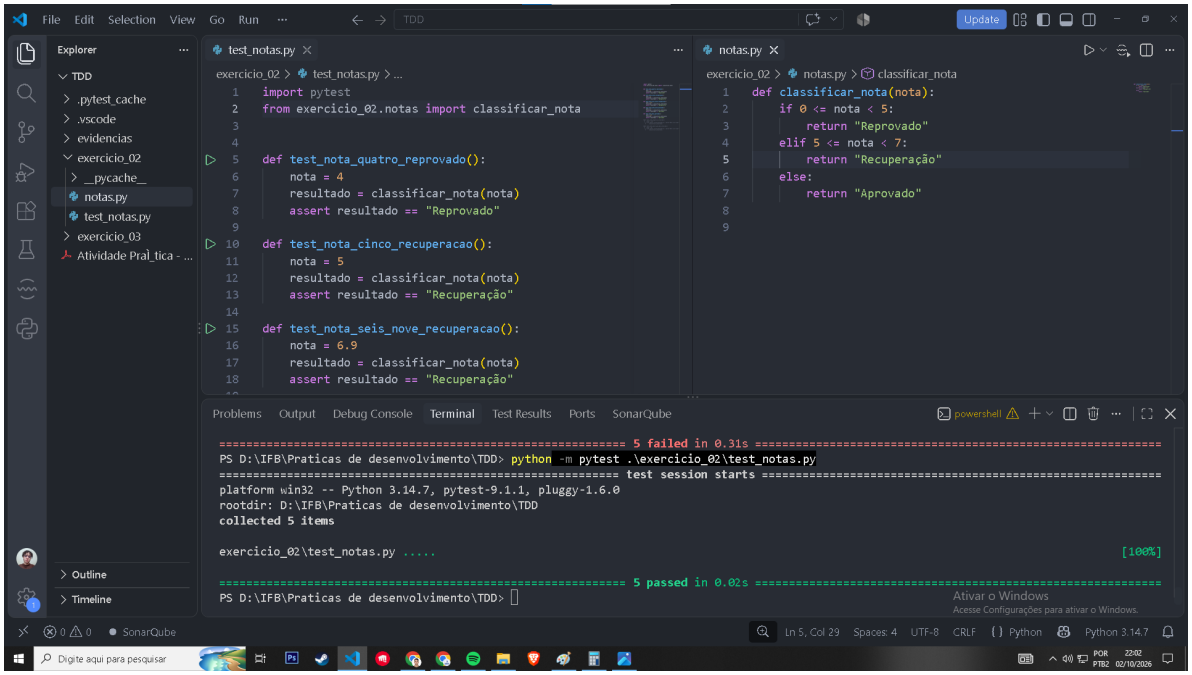


Figura 5: Evidência da fase GREEN: bateria com os 5 casos de teste de classificação obrigatórios aprovados.

3. **Como você tratou notas inválidas?**

   O tratamento de entradas inválidas (valores estritamente inferiores a zero ou superiores a dez) foi feito via lançamento explícito da exceção `ValueError` seguida da descrição `"Nota inválida"`. Seguindo o fluxo TDD, foram criados previamente os testes com `with pytest.raises(ValueError, match="Nota inválida")`, observada a sua falha e, a seguir, criada a cláusula de guarda no módulo `notas.py`. A execução confirmou a passagem integral da suíte de testes completa.

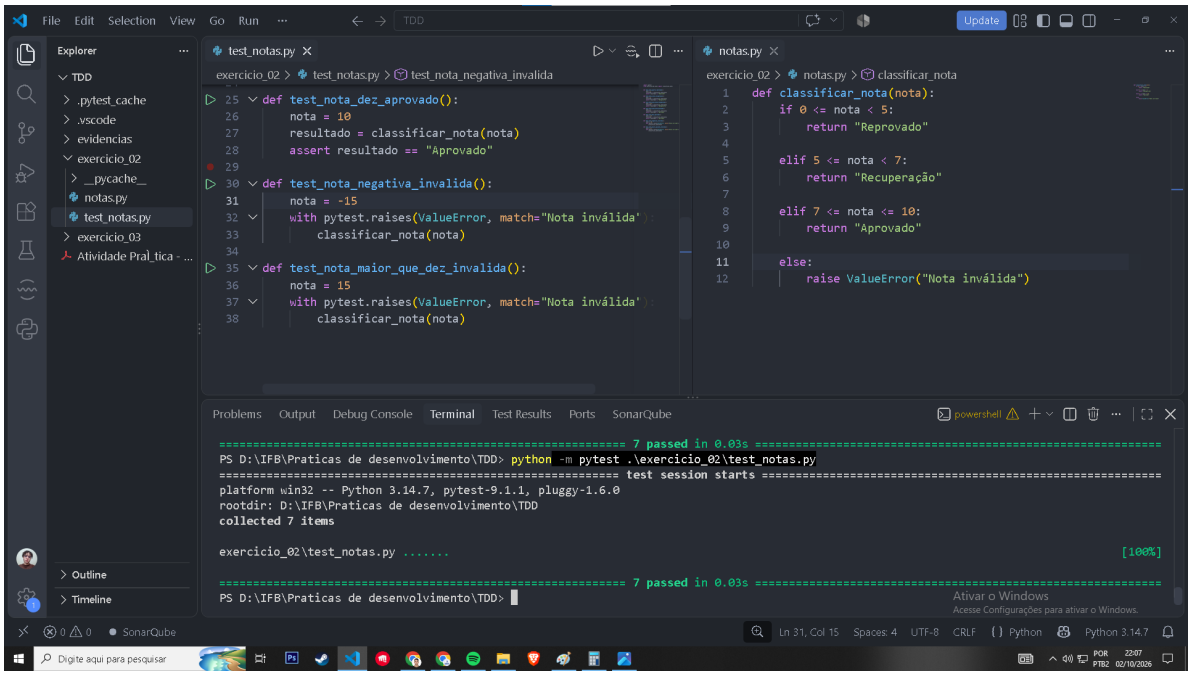


Figura 6: Evidência da fase REFACTOR: suíte completa com os 7 testes executados com sucesso no Pytest.

Exercício 3: Carrinho de Compras

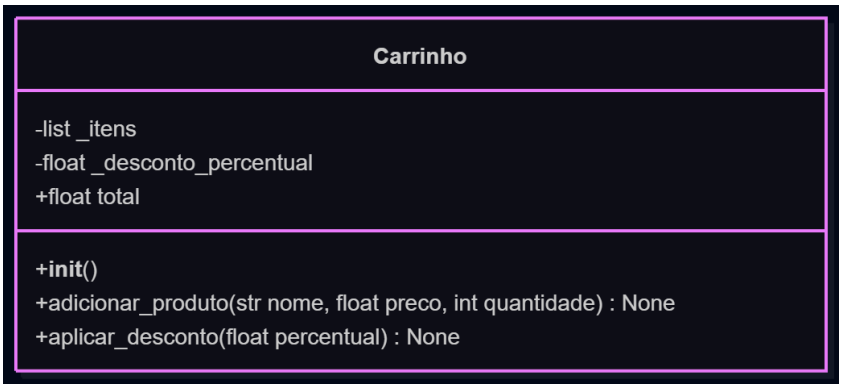


Figura 7: Modelagem de classes UML da entidade Carrinho.

1. **Quais funcionalidades foram implementadas?**

   Foi desenvolvida a classe `Carrinho` com suporte completo às seguintes operações de negócio:

   - Inicialização do estado interno com uma lista de itens vazia e percentual de desconto zerado.
   - Adição de itens unitários ou múltiplos informando nome, preço e parâmetro opcional de quantidade (com valor padrão unitário).
   - Validação defensiva de entrada: rejeição de quantidade menor ou igual a zero com lançamento da exceção `ValueError("Quantidade inválida")`.
   - Aplicação de desconto percentual acumulável sobre o valor total do carrinho.
   - Cálculo dinâmico do montante total via método `total()`, que percorre os itens acumulando os subtotais e aplicando o abatimento percentual configurado.

2. **Quais testes foram criados?**

   No módulo `test_carrinho.py`, foram implementados 6 casos de teste automatizados com o *Pytest* para validar as regras obrigatórias:

- `teste_carrinho_vazio()`: confirma que uma nova instância do carrinho inicia com total igual a 0.
- `teste_produto_100_total_100()`: verifica a inserção de um único produto (Mouse de R$ 100,00) totalizando R$ 100,00.
- `teste_dois_produtos_total_250()`: valida a inserção cumulativa de dois produtos distintos (Mouse de R$ 100,00 e Teclado de R$ 150,00), totalizando R$ 250,00.
- `teste_produtos_com_quantidade_total_300()`: testa o parâmetro explícito de quantidade (Mouse de R$ 100,00 com quantidade 3), resultando no total de R$ 300,00.
- `teste_total_500_10_porcento_desconto()`: valida o abatimento percentual de 10% sobre um carrinho de R$ 500,00, confirmando o total final de R$ 450,00.
- `teste_quantidade_invalida()`: valida o fluxo de exceção ao tentar inserir um produto com quantidade igual a 0, assegurando o disparo de `ValueError` via `pytest.raises`.

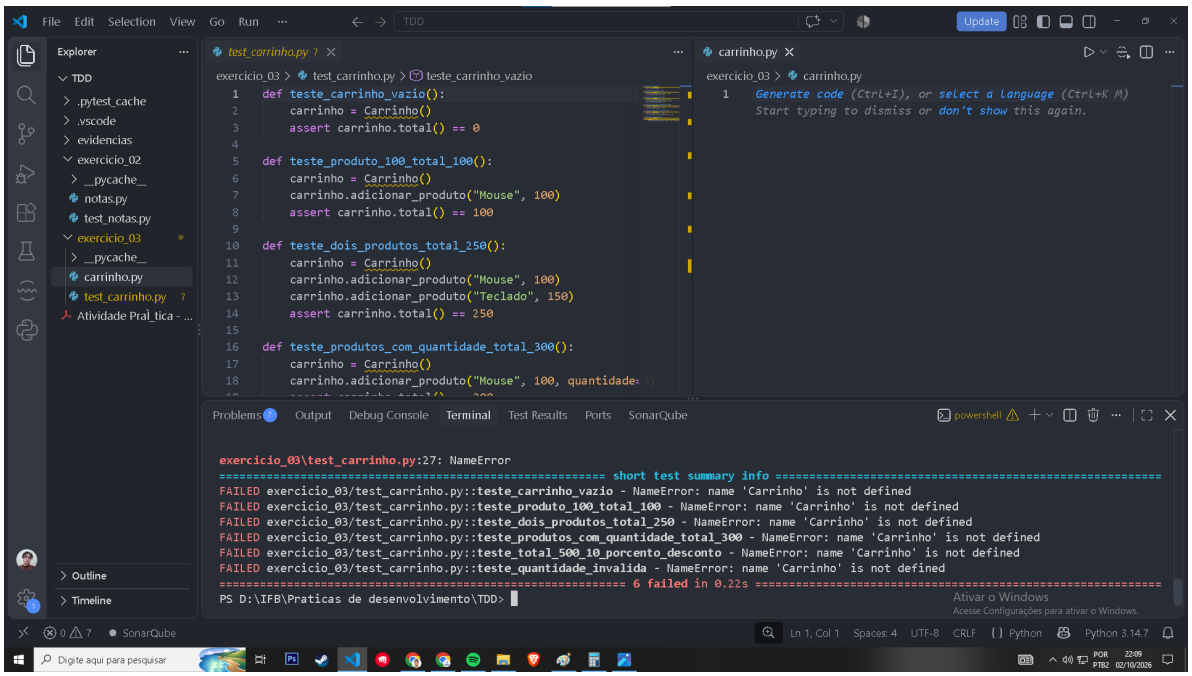


Figura 8: Evidência da fase RED: execução prévia dos testes acusando NameError pela ausência da classe.

3. **Qual foi a principal dificuldade encontrada durante o desenvolvimento?**

   A principal dificuldade foi a utilização do Python para a definição de classe. Questões como a obrigatoriedade da referência explícita `self` nos métodos de instância, o controle estrito de indentação de blocos e a manipulação de estruturas de dados dinâmicas (dicionários e listas por compreensão no cálculo do somatório) exigiram atenção. Além disso, compreender o ciclo do TDD na modelagem de classes — garantindo que a interface pública dos métodos fosse planejada primeiro a partir do teste unitário antes de qualquer definição da estrutura interna da classe — exigiu uma mudança de raciocínio em relação ao desenvolvimento tradicional.

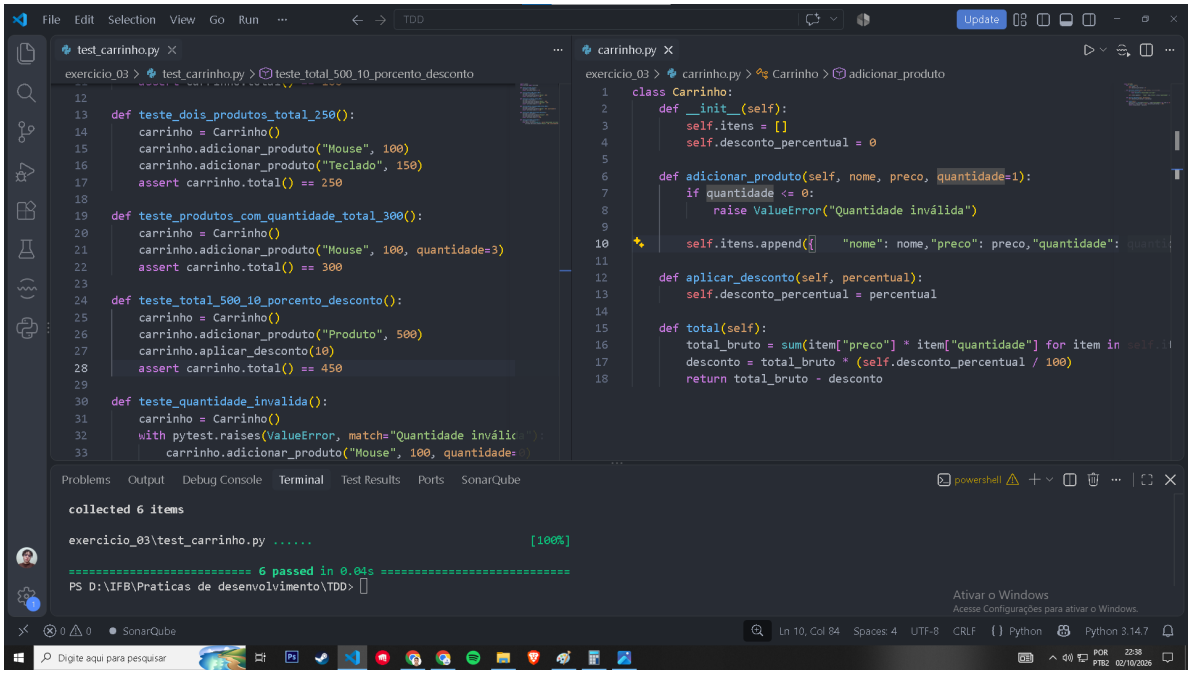


Figura 9: Evidência da fase GREEN: implementação da classe satisfazendo todos os 6 testes no Pytest.

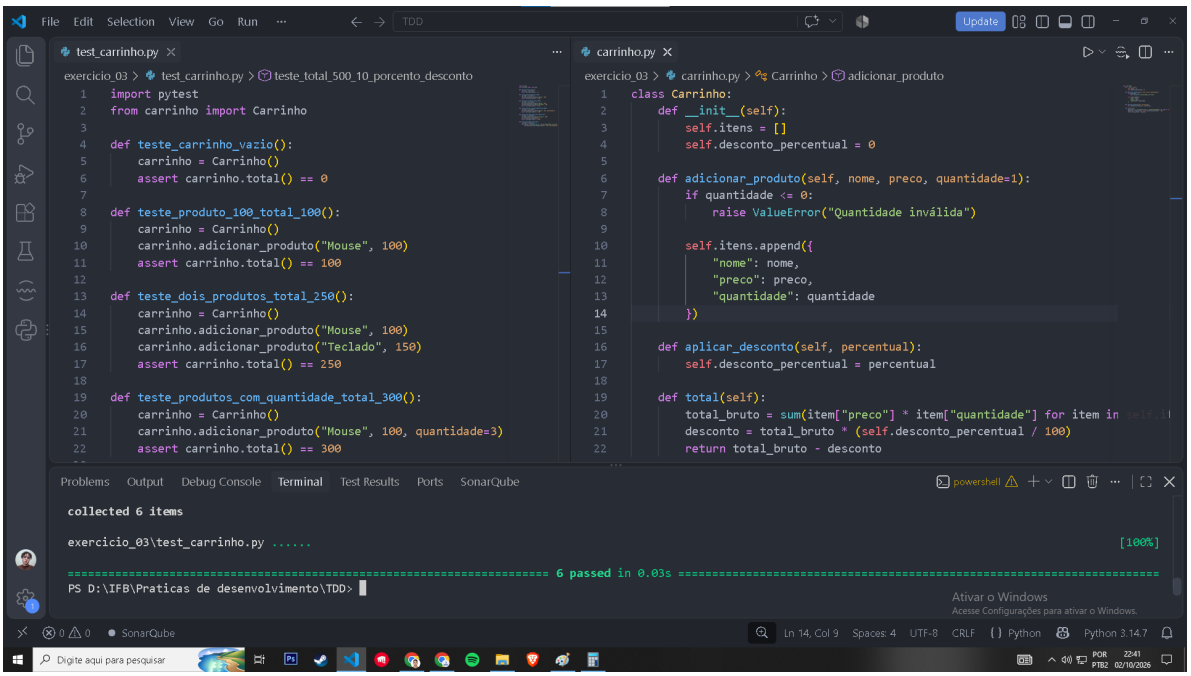


Figura 10: Evidência da fase REFACTOR: suíte integral com os 6 testes passando com código consolidado.

Reflexão final:

- Explique, com suas próprias palavras, o que significa RED → GREEN → REFACTOR.

  Prática de desenvolvimento de sistema onde primeiro se criam os testes, antes do código funcional, com a intenção de que não passem (RED), logo após isso, desenvolve-se o código mínimo para que os testes passem (Green) e depois se melhora o código em organização e legibilidade, por exemplo(Refactor).

- Qual é a principal diferença entre desenvolver primeiro o código e depois criar os testes e utilizar TDD?

  Escrever os testes antes cria uma noção prévia do que cada método deve fazer e as limitações a considerar, incluindo exceções e tratamento de erros.